
CONCEPTION D'UN PROCESSEUR : CHEMIN DE DONNÉES

Objectif

L'objectif de ce TP est de concevoir un "processeur fonctionnel" capable de :

- Charger des instructions depuis une RAM d'instructions ;
- Exécuter une suite d'instructions en respectant le cycle **Fetch-Decode-Execute** ;
- Gérer les instructions arithmétiques, **LOAD**, **CST**, et **STOR**.

Exception : Les instructions de type **JUMP** seront traitées dans le prochain TP.

N'hésitez pas pour ce TP à vous referez aux schémas disponibles dans les supports de cours ou dans le manuel !

Étapes d'implémentation

Nous vous proposons de suivre les différentes étapes proposées ici pour implémenter votre processeur.

Remarque sur l'adressage du PC Dans notre conception, le **PC** conserve l'adresse de la **dernière instruction fetchée**, ce qui garantit la cohérence entre les registres **PC** et **IF** lors d'une pause de la simulation. Ainsi, la **première adresse fetchée** sera 0x01 : l'instruction à l'adresse 0x00 ne sera jamais exécutée au démarrage.

Pour éviter tout problème, le compilateur **CYT-VX8** (script Python) ajoute systématiquement une instruction **NOP** au début du code à l'adresse 0x00. —

Gestion d'une suite d'instructions arithmétiques

A l'issue de cette étape, vous serez capable d'exécuter une suite d'opérations réalisée par l'ALU (instructions **ADD SUB CMP AND ...**).

- **Ajouter les RAM** :
 - Utilisez une **RAM d'instructions** capable de stocker **24 bits par adresse** (pour les instructions du CYT-VX8).
 - Utilisez une **RAM de données** pour stocker des **char signés (8 bits)**, avec un **adressage sur 8 bits** (256 adresses max).
 - Vous pouvez utiliser les RAM déjà disponibles sous Logisim.
- **Implémenter le IP et son incrémenteur** :
 - Créez un **registre IP de 24 bits** pour stocker l'adresse de l'instruction courante.
 - Ajoutez un **incrémenteur** pour faire +1 sur le contenu du IP à chaque cycle d'horloge.
- **Brancher le module (IP + incrémenteur) à la RAM d'instructions** :
 - Connectez la sortie du module IP+incrémenteur à l'entrée d'adresse de la **RAM d'instructions**.
 - Remplissez quelques adresses de la RAM avec des instructions arithmétiques (ex : **ADD R1, R2, R3** à l'adresse 0x00, **SUB R4, R5, R6** à 0x01, **NOT R4, R5** à 0x02).
 - Vérifiez que les instructions sont bien chargées dans le bon ordre à chaque cycle.
- **Ajouter le module FETCH** : Ajoutez le registre **IF** de 24 bits en sortie de la RAM d'instructions.
- Brancher le module **FETCH** au décodeur que vous avez implémenté au TP précédent

Test final :

- Votre processeur devrait maintenant pouvoir exécuter **plusieurs instructions arithmétiques à la suite** !
- Testez avec une séquence simple.

Instructions **LOAD** et **CST**

Avec l'ajout de ces instructions, les registres de données pourront sauvegarder :

- Des valeurs issues de l'ALU (normalement déjà le cas) ;
- Des constantes données directement dans l'instruction (voir instruction **CNST** dans le manuel) ;
- Des valeurs provenant de la RAM de données (voir instruction **LOAD** dans le manuel).

Puisque les entrées des registres peuvent provenir de **plusieurs sources différentes**, il faudra :

- **Ajouter un multiplexeur en entrée des registres** : Ce multiplexeur permettra de sélectionner la source des données à écrire dans les registres.
- **Modifier le décodeur** : Le décodeur doit être modifié pour :
 - Générer les bits de sélection du nouveau multiplexeur ;
 - Récupérer correctement la constante (pour **CNST**) ou la valeur de la RAM (pour **LOAD**) à stocker dans le registre.
-

Instruction STOR

Cette instruction permet de stocker la valeur d'un des registre de données dans la RAM de données, à une adresse données dans l'instruction. Vérifier l'opcode de l'instruction sur le manuel et implementer la !